

TESTING

Strategia di Test/Test Plan

Il team ha attuato una strategia di test White Box per verificare le funzionalità implementate e il codice interno.

L'attività di testing è stata suddivisa in:

- Unit Testing: utilizzo di JUnit5 per testare logica di business e validazione dei dati, sviluppo di test automatizzati per componenti.

Di seguito un estratto esempio di test per la verifica dei dati in fase di registrazione:

```
@Test
void testVerificaDati_CampiErrati() {
    String result = DataValidator.verificaDati("", "Rossi", "user", "a@b.c", "P1!", "P1!");
    assertEquals("Tutti i campi sono obbligatori!", result);

    result = DataValidator.verificaDati("Mario", "", "user", "a@b.c", "P1!", "P1!");
    assertEquals("Tutti i campi sono obbligatori!", result);

    result = DataValidator.verificaDati("Mario", "Rossi", "mario.rossi", "a@b.c", "P1!", "P1!");
    assertEquals("L'username non è valido! Non utilizzare simboli speciali: sono possibili solo trattini (-) e underscore (_)!", result);
}
```

- Manual Testing: test manuale in seguito ad ogni modifica effettuata e prima di ogni commit. Test effettuati tramite interfaccia Vaadin navigando tra le viste.

Di seguito un esempio di test manuale:

- Accedere come Utente
- Selezionare colonnina
- Prenotare slot
- Risultato atteso: la prenotazione appare nella pagina apposita
- Esito: Passato

Dettaglio dei Test Implementati

Classe di Test	Classe Testata	Obiettivo Test
DataValidatorTest	DataValidator	Validazione dati inseriti in input • Campi nulli o vuoti • Password non coincidenti • Regex username e e-mail
AutoTest	Auto	Verifica inizializzazione stato auto e percentuale carica in base al tipo di automobile Inizializzazione coerente dello stato della carica.

Classe di Test	Classe Testata	Obiettivo Test
PrenotazioneTest	Prenotazione	Verifica corretta inizializzazione dello stato della prenotazione.
TipoAutoTest	TipoAuto (Enum)	Verifica conversione Stringa-Enumerativo anche in caso di input minuscolo/maiuscolo
QRCodeTest	QRCode	Verifica generazione QR code corretta senza eccezioni tramite il metodo generaQR

Code Coverage

È stata effettuata un'analisi della copertura utilizzando il tool Eclemma. La strategia di testing ha evitato le classi views e database, concentrandosi maggiormente su pacchetti utils, models e enums.

Qui la percentuale di istruzioni verificata dai test per le classi analizzate.

Classe Critica	Copertura	Percentuale Eclemma
DataValidator	58,9%	 DataValidator.java  58,9 %
Auto	62,2%	 Auto.java  62,2 %
Prenotazione	68,2%	 Prenotazione.java  68,2 %
TipoAuto	96,4%	 TipoAuto.java  96,4 %
QRCode	85,7%	 QRCode.java  85,7 %

Le percentuali riflettono una strategia di testing focalizzata su metodi con maggior rischio di errore, evitando test su metodi secondari, getter e setter.

Questo riduce la percentuale di copertura di alcune classi, ma mantenendo la copertura delle regole e delle parti critiche del codice.

Analizzando la branch-coverage otteniamo:

Classe Critica	Copertura	Descrizione
DataValidator	75 %	Coperti rami decisionali
Auto	100%	Copertura totale
Prenotazione	N/A	Assenza di rami
TipoAuto	N/A	Test su enumerativi
QRCode	N/A	Esecuzione lineare